ASSIGNMENT 32

Project 1: E-commerce Data Warehouse

Create_tables.sql

```
CREATE DATABASE ecommerce_dw;
USE ecommerce_dw;

-- Customer Dimension
CREATE TABLE customer (
  customer_id INT PRIMARY KEY,
  name VARCHAR(100),
  city VARCHAR(50)
);

-- Product Dimension
CREATE TABLE product (
  product_id INT PRIMARY KEY,
  name VARCHAR(100),
  category VARCHAR(50)
);

-- Time Dimension
CREATE TABLE time_dim (
  time_id INT AUTO_INCREMENT PRIMARY KEY,
  date DATE,
  month INT,
  year INT
);
```

```
-- Fact Table
CREATE TABLE sales (
  sale_id INT PRIMARY KEY,
  customer_id INT,
  product_id INT,
  time_id INT,
  amount DECIMAL(10,2),
  quantity INT,
  FOREIGN KEY (customer_id) REFERENCES customer(customer_id),
  FOREIGN KEY (product_id) REFERENCES product(product_id),
  FOREIGN KEY (time_id) REFERENCES time_dim(time_id)
);
```

queries.sql

```
-- Monthly Sales
SELECT t.month, SUM(s.amount) AS total_sales
FROM sales s
JOIN time_dim t ON s.time_id = t.time_id
GROUP BY t.month;

-- Top Customers
SELECT c.name, SUM(s.amount) AS total_spent
FROM sales s
JOIN customer c ON s.customer_id = c.customer_id
GROUP BY c.name
ORDER BY total_spent DESC;

-- Best Selling Products
```

```
SELECT p.name, SUM(s.quantity) AS total_sold
FROM sales s
JOIN product p ON s.product_id = p.product_id
GROUP BY p.name
ORDER BY total_sold DESC;
```

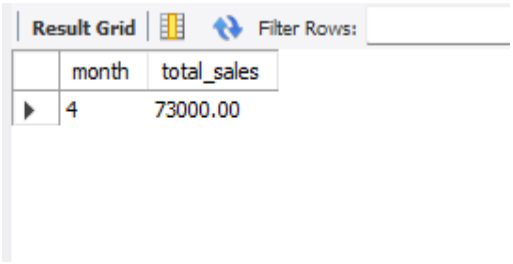
etl.py

```
import pandas as pd
import mysql.connector

# DB Connection
conn = mysql.connector.connect(
  host="localhost",
  user="root",
  password="1234",
  database="ecommerce_dw"
)
cursor = conn.cursor()

# Load CSV files
customers = pd.read_csv("./data/customers.csv")
products = pd.read_csv("./data/products.csv")
sales = pd.read_csv("./data/sales.csv")
for _, row in customers.iterrows():
  cursor.execute(
    "INSERT INTO customer (customer_id, name, city) VALUES (%s, %s, %s)",
    (row['customer_id'], row['name'], row['city'])
  )
```

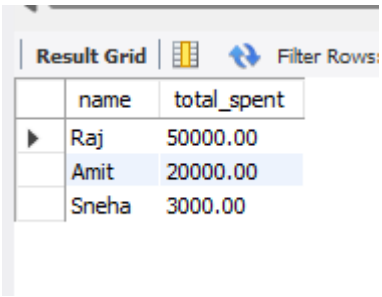
```
for _, row in products.iterrows():
    cursor.execute(
        "INSERT INTO product (product_id, name, category) VALUES (%s, %s, %s)",
        (row['product_id'], row['name'], row['category'])
    )
time_data = []
for date in sales['date'].unique():
    dt = pd.to_datetime(date)
    time_data.append((dt.strftime("%Y-%m-%d"), dt.month, dt.year))

for t in time_data:
    cursor.execute(
        "INSERT INTO time_dim (date, month, year) VALUES (%s, %s, %s)",
        t
    )
cursor.execute("SELECT * FROM time_dim")
time_rows = cursor.fetchall()
time_map = {
    row[1].strftime("%Y-%m-%d") if hasattr(row[1], 'strftime') else str(row[1]): row[0]
    for row in time_rows
}
for _, row in sales.iterrows():
    lookup_date = pd.to_datetime(row['date']).strftime("%Y-%m-%d")
    time_id = time_map[lookup_date]
    cursor.execute(
        "INSERT INTO sales (sale_id, customer_id, product_id, time_id, amount, quantity)
VALUES (%s,%s,%s,%s,%s,%s)",
        (row['sale_id'], row['customer_id'], row['product_id'],
```

```
        time_id, row['amount'], row['quantity'])
    )

conn.commit()
cursor.close()
conn.close()

print("ETL Completed Successfully!")
```

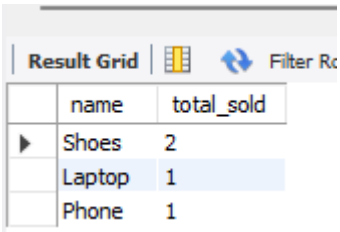
output:



month	total_sales
4	73000.00



name	total_spent
Raj	50000.00
Amit	20000.00
Sneha	3000.00



name	total_sold
Shoes	2
Laptop	1
Phone	1

Project 2: Hospital Management Analytics

create_tables.sql

```
CREATE DATABASE hospital_db;
USE hospital_db;
```

```
-- Patients Table
CREATE TABLE patients (
  patient_id INT PRIMARY KEY,
  name VARCHAR(100),
  age INT,
  gender VARCHAR(10)
);

-- Doctors Table
CREATE TABLE doctors (
  doctor_id INT PRIMARY KEY,
  name VARCHAR(100),
  specialization VARCHAR(50)
);

-- Appointments Table (FACT TABLE)
CREATE TABLE appointments (
  appointment_id INT PRIMARY KEY,
  patient_id INT,
  doctor_id INT,
  appointment_date DATE,
  fee DECIMAL(10,2),
  FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
  FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
);
```

insert_data.sql

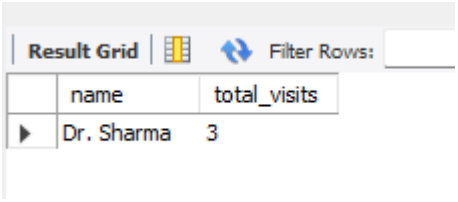
```
-- Patients
INSERT INTO patients VALUES
(1, 'Raj', 22, 'Male'),
(2, 'Amit', 30, 'Male'),
(3, 'Sneha', 25, 'Female');

-- Doctors
INSERT INTO doctors VALUES
(101, 'Dr. Sharma', 'Cardiology'),
(102, 'Dr. Mehta', 'Dermatology'),
(103, 'Dr. Rao', 'General');

-- Appointments
INSERT INTO appointments VALUES
(1, 1, 101, '2026-04-01', 500),
(2, 2, 101, '2026-04-01', 500),
(3, 3, 102, '2026-04-02', 300),
(4, 1, 103, '2026-04-02', 200),
(5, 2, 101, '2026-04-03', 500);
```

queries.sql

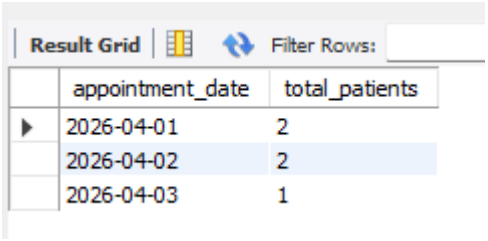
```
-- Most visited doctor
SELECT d.name, COUNT(*) AS total_visits
FROM appointments a
JOIN doctors d ON a.doctor_id = d.doctor_id
GROUP BY d.name
ORDER BY total_visits DESC
```

```
LIMIT 1;

-- Daily Patient Count
SELECT appointment_date, COUNT(DISTINCT patient_id) AS total_patients
FROM appointments
GROUP BY appointment_date;

-- Revenue Analysis
SELECT SUM(fee) AS total_revenue
FROM appointments;

-- Revenue Per Doctor
SELECT d.name, SUM(a.fee) AS revenue
FROM appointments a
JOIN doctors d ON a.doctor_id = d.doctor_id
GROUP BY d.name
ORDER BY revenue DESC;

-- Most Common Specialization
SELECT d.specialization, COUNT(*) AS total_visits
FROM appointments a
JOIN doctors d ON a.doctor_id = d.doctor_id
GROUP BY d.specialization
ORDER BY total_visits DESC;
```

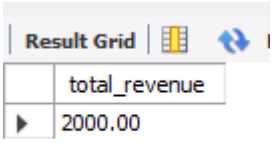
Output:



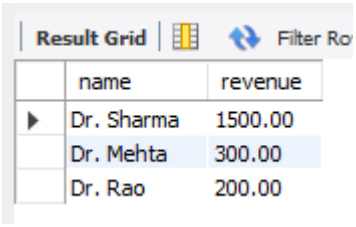
name	total_visits
Dr. Sharma	3



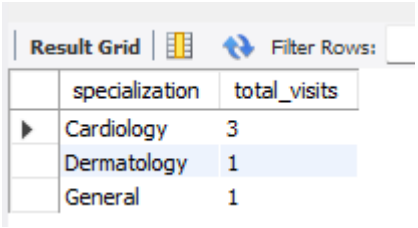
appointment_date	total_patients
2026-04-01	2
2026-04-02	2
2026-04-03	1



total_revenue
2000.00



name	revenue
Dr. Sharma	1500.00
Dr. Mehta	300.00
Dr. Rao	200.00



specialization	total_visits
Cardiology	3
Dermatology	1
General	1

Project 3: Social Media (NoSQL)

sample_data.json

```
[
 {
   "user_id": 1,
   "name": "Raj",
   "email": "raj@gmail.com",
   "posts": [
```

```
    {
      "post_id": 101,
      "content": "Hello World",
      "likes": 10,
      "comments": [
       {
         "user": "Amit",
         "text": "Nice post",
         "replies": [
          {
            "user": "Sneha",
            "text": "Agree!"
          }
         ]
       }
      ]
    }
  ]
},
{
  "user_id": 2,
  "name": "Amit",
  "email": "amit@gmail.com",
  "posts": [
   {
     "post_id": 102,
     "content": "MongoDB is cool",
     "likes": 25,
```

```
      "comments": []
    }
  ]
 }
]
```

queries.js

```
// Most Liked Posts
db.users.aggregate([
 { $unwind: "$posts" },
 { $sort: { "posts.likes": -1 } },
 { $limit: 5 },
 {
  $project: {
   user: "$name",
   post: "$posts.content",
   likes: "$posts.likes"
  }
 }
])

// Active Users (Most Posts)
db.users.aggregate([
 {
  $project: {
   name: 1,
   total_posts: { $size: "$posts" }
  }
```

```
 },
 { $sort: { total_posts: -1 } }
])

// Posts with Comments Count
db.users.aggregate([
 { $unwind: "$posts" },
 {
  $project: {
   post: "$posts.content",
   comment_count: { $size: "$posts.comments" }
  }
 }
])

// Find Posts with More Than 10 Likes
db.users.find(
 { "posts.likes": { $gt: 10 } },
 { name: 1, "posts.content": 1, "posts.likes": 1 }
)
```

Output:

```
> db.users.find(
    { "posts.likes": { $gt: 10 } },
    { name: 1, "posts.content": 1, "posts.likes": 1 }
  )
< {
    _id: ObjectId('69d666a6740a3a15ffede5e4'),
    name: 'Amit',
    posts: [
      {
        content: 'MongoDB is cool',
        likes: 25
      }
    ]
  }
```

```
> use social_media_db
< switched to db social_media_db
> db.users.aggregate([
    { $unwind: "$posts" },
    { $sort: { "posts.likes": -1 } },
    { $limit: 5 },
    {
      $project: {
        user: "$name",
        post: "$posts.content",
        likes: "$posts.likes"
      }
    }
  ])
< {
    _id: ObjectId('69d666a6740a3a15ffede5e4'),
    user: 'Amit',
    post: 'MongoDB is cool',
    likes: 25
  }
  {
    _id: ObjectId('69d666a6740a3a15ffede5e3'),
    user: 'Raj',
    post: 'Hello World',
    likes: 10
  }
```

```
> db.users.aggregate([
    {
      $project: {
        name: 1,
        total_posts: { $size: "$posts" }
      }
    },
    { $sort: { total_posts: -1 } }
  ])
< {
    _id: ObjectId('69d666a6740a3a15ffede5e3'),
    name: 'Raj',
    total_posts: 1
  }
  {
    _id: ObjectId('69d666a6740a3a15ffede5e4'),
    name: 'Amit',
    total_posts: 1
  }
> db.users.aggregate([
    { $unwind: "$posts" },
    {
      $project: {
        post: "$posts.content",
        comment_count: { $size: "$posts.comments" }
      }
    }
  ])
< {
    _id: ObjectId('69d666a6740a3a15ffede5e3'),
    post: 'Hello World',
    comment_count: 1
  }
  {
    _id: ObjectId('69d666a6740a3a15ffede5e4'),
    post: 'MongoDB is cool',
    comment_count: 0
  }
```

Project 4: Log Analytics System

logs.json

```
[
 {
   "timestamp": "2026-04-08T10:00:00",
```

```
  "level": "INFO",
  "service": "auth",
  "message": "User login successful"
 },
 {
  "timestamp": "2026-04-08T10:05:00",
  "level": "ERROR",
  "service": "payment",
  "message": "Payment failed"
 },
 {
  "timestamp": "2026-04-08T11:00:00",
  "level": "ERROR",
  "service": "auth",
  "message": "Invalid token"
 },
 {
  "timestamp": "2026-04-09T09:00:00",
  "level": "INFO",
  "service": "order",
  "message": "Order placed"
 }
]
```

queries.js

```
// Error Frequency
db.logs.countDocuments({ level: "ERROR" })


// Logs Per Day
```

```
db.logs.aggregate([
 {
  $group: {
   _id: {
    $dateToString: { format: "%Y-%m-%d", date: { $toDate: "$timestamp" } }
   },
   total_logs: { $sum: 1 }
  }
 }
])

// Errors Per Service
db.logs.aggregate([
 { $match: { level: "ERROR" } },
 {
  $group: {
   _id: "$service",
   error_count: { $sum: 1 }
  }
 }
])

// Logs in Last 24 Hours
db.logs.find({
 timestamp: {
  $gte: new Date(Date.now() - 24 * 60 * 60 * 1000)
 }
})
```

```
// Most Active Service
db.logs.aggregate([
 {
  $group: {
   _id: "$service",
   total: { $sum: 1 }
  }
 },
 { $sort: { total: -1 } },
 { $limit: 1 }
])
```

Output:

```
> use log_db
< switched to db log_db
> db.logs.createIndex({ timestamp: 1 })
< timestamp_1
> db.logs.createIndex({ level: 1 })
< level_1
> db.logs.createIndex({ service: 1 })
< service_1
> db.logs.countDocuments({ level: "ERROR" })
< 2
> db.logs.aggregate([
    {
      $group: {
        _id: {
          $dateToString: { format: "%Y-%m-%d", date: { $toDate: "$timestamp" } }
        },
        total_logs: { $sum: 1 }
      }
    }
  ])
< {
    _id: '2026-04-08',
    total_logs: 3
  }
  {
    _id: '2026-04-09',
    total_logs: 1
  }
```

```
> db.logs.aggregate([
    { $match: { level: "ERROR" } },
    {
      $group: {
        _id: "$service",
        error_count: { $sum: 1 }
      }
    }
  ])
< {
    _id: 'payment',
    error_count: 1
  }
  {
    _id: 'auth',
    error_count: 1
  }
> db.logs.find({
    timestamp: {
      $gte: new Date(Date.now() - 24 * 60 * 60 * 1000)
    }
  })
<
> db.logs.aggregate([
    {
      $group: {
        _id: "$service",
        total: { $sum: 1 }
      }
    },
    { $sort: { total: -1 } },
    { $limit: 1 }
  ])
< {
    _id: 'auth',
    total: 2
  }
```

CASE STUDY: Millions of Transactions Daily

1. Use BOTH (Hybrid Approach), Use SQL for transactional systems and NoSQL for scalable analytics workloads.
2. Design a star schema with a central fact table and multiple dimension tables for efficient analytical queries.
3. Use Python-based ETL for data processing and Airflow for scheduling, with Spark for handling large-scale data.
4. Performance can be improved using indexing, partitioning, caching, and distributed processing with Spark.